04-630: Data Structures and Algorithms for Engineers

Assignment III: Text Analysis Using Binary Search Trees

Rachel Tuyishimire

Rtuyishi

Reflection Report

The project used a Binary Search Tree (BST) to analyze text by counting word frequencies, tracking tree levels, and measuring probe steps. To avoid memory leaks, the tree was deleted after each file using a deleteTree function. Words were stored in fixed-size character arrays (50 characters), which helped reduce memory use but could cut off longer words.

Large text files, including those with over 10,000 words, were processed one word at a time. This kept memory use below 1 MB and supported scalability. Operations like inserting or searching for words in the BST depend on the tree's height efficient in balanced trees (O(log n)) but slower in skewed trees (O(n)). In-order traversal provided alphabetically sorted output. Balancing the tree was not included to keep the design simple.

Each node held the word, its frequency, and its level in the tree. The program was tested with repeated words, punctuation-heavy inputs, different letter cases, empty files, single-word files, and punctuation-only files. An issue was found with apostrophes, which caused words like "let" and "let's" to be counted separately. This was fixed by trimming words at the apostrophe.

Punctuation was handled using the strtok function. Memory concerns with large files were solved by processing input sequentially. Other data structures were considered. Hash tables offered faster lookups but required extra sorting. Tries supported prefix matching but were avoided due to added complexity.

Possible improvements include adding tree balancing for better performance and exploring tries for large word sets. With careful testing and memory handling, the BST provided accurate and efficient text analysis that met the assignment goals.